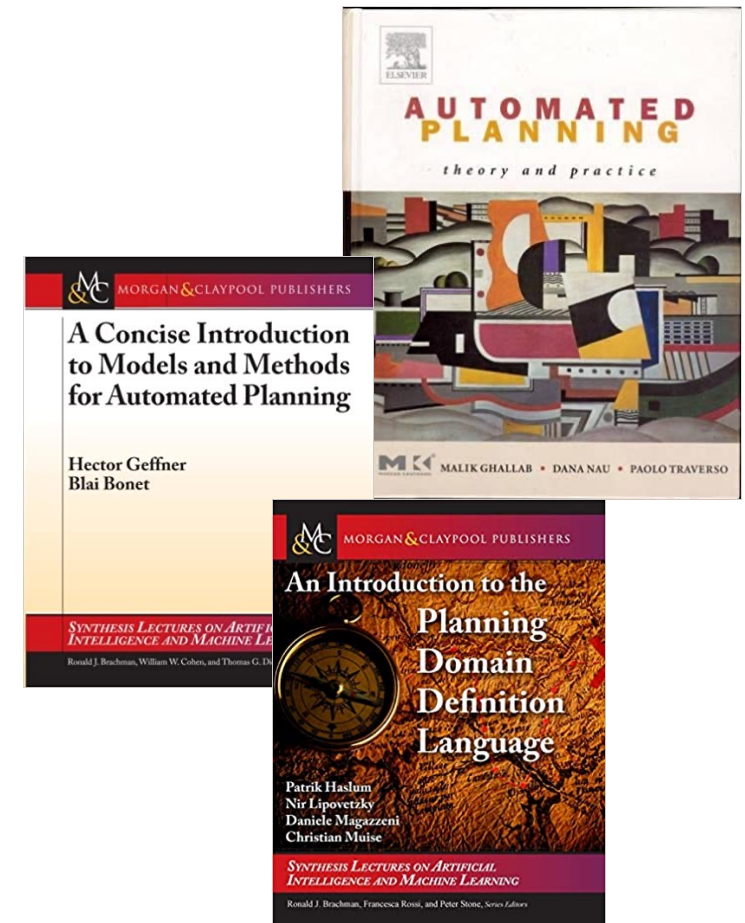


Planning: Model the Environment as a Transition System

Planning:

- Inherit from KR the idea that environment can be represented in terms of
 - Preconditions for agent actions
 - Effects of agent actions + solution to “Frame Problem”
- But use them as a specification for generating a transition system
 - a single model vs a theory
- Reasoning
 - based on “model checking”
- Two notable cases:
 - Classical planning: everything determined by agent actions
 - Planning in nondeterministic domains (FOND):
 - Agent: instructs actions
 - Env: determine their effects



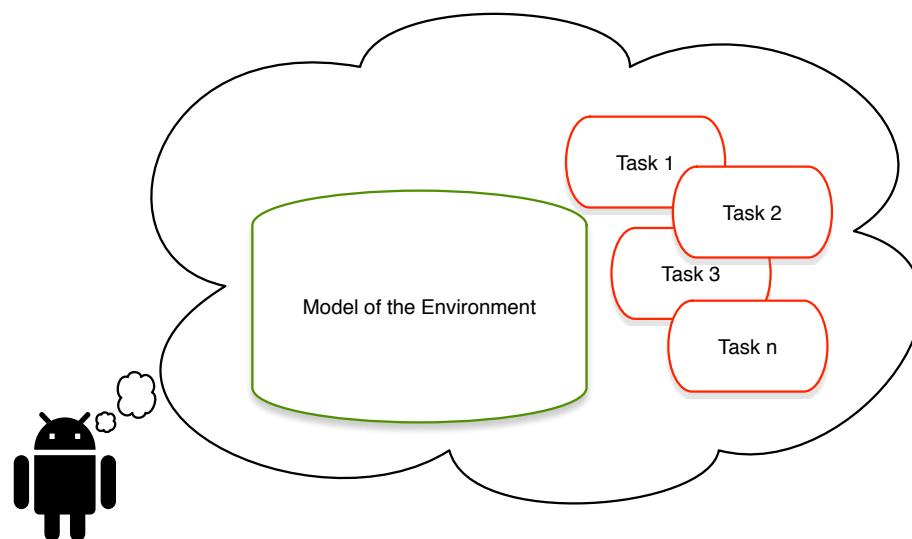
Focus on Terminating Tasks

Planning in AI:

- Is all about having a task specification or “goal” and producing a “plan” (or strategy or policy) to satisfy the task in the environment model.
- Which tasks?
 - A **task that terminates!**
 - Typically, just reaching a certain state in the environment

Why tasks that terminates?

- Because it is the agent that is planning/reasoning
- If the task would not terminate, the agent would be stuck into doing the same task forever
- But then, why bother with equipping it with a model of the environment and of the task at all?
- Note it is the agent, NOT the designer, who has such models

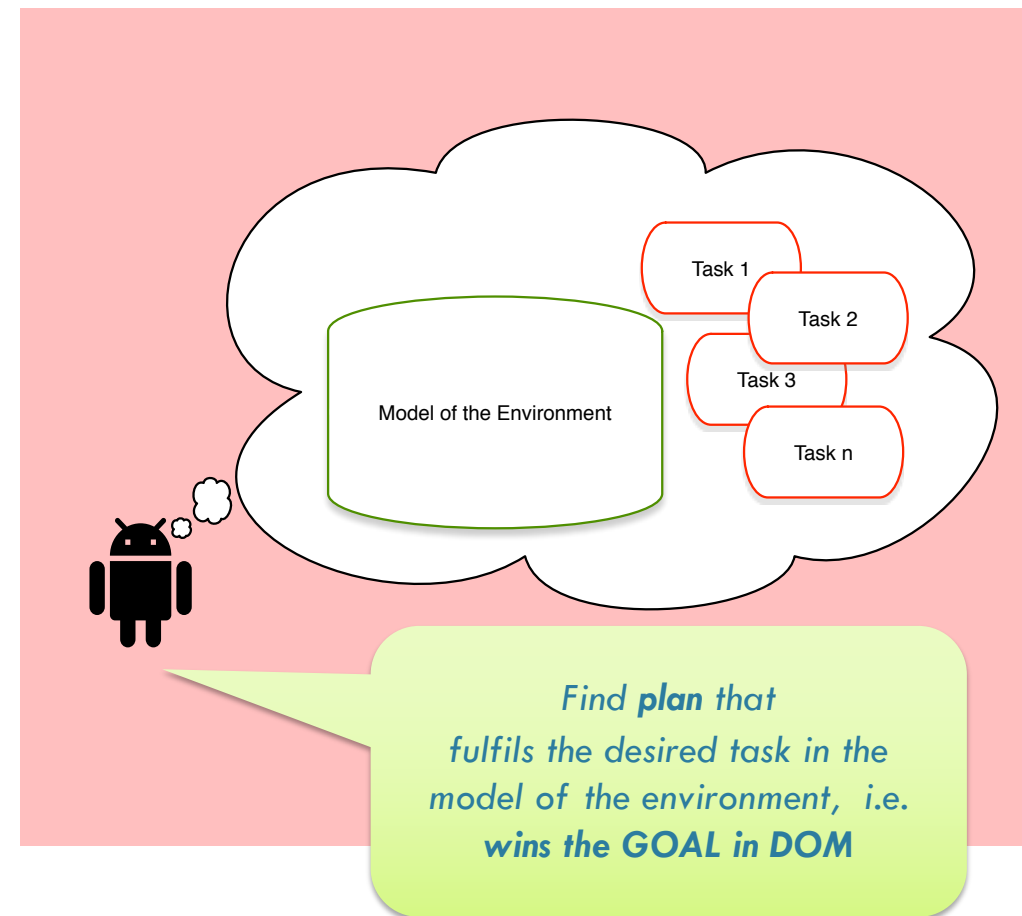


Planning in AI

PLANNING

Domains and Goals

- **Environment Model (DOM)**
 - Spec. of **environment's behaviors** of the world in response to **agent's action**
 - **Domain** expressed as with specific formalisms
 - STRIPS
 - ADL
 - PDDL
 - **Domain** is (or better generates) a **deterministic transition system**
- **Agent's Task (GOAL)**
 - Spec. of **agent's task**
 - **GOAL** expressed as **reaching a state of the domain with desired properties**
- **Find agent's plan/behavior/policy/strategy** that fulfills **GOAL** in **DOM**



Planning in Deterministic Domains

It's a transition system!

Deterministic domain (including initial state)

$\mathcal{D} = (2^{\mathcal{F}}, \mathcal{A}, s_0, \delta, \alpha)$ where:

- $\mathcal{F}$ **fluents** (atomic propositions)
- $\mathcal{A}$ **actions** (atomic symbols)
- $2^{\mathcal{F}}$ set of states
- s_0 initial state (initial assignment to fluents)
- $\alpha(s) \subseteq \mathcal{A}$ represents **action preconditions**
- $\delta(s, a) = s'$ with $a \in \alpha(s)$ represents **action effects (including frame)**.

Traces

A **trace** for $\mathcal{D}$ is a finite sequence:

$$s_0, a_1, s_1, \dots, a_n, s_n$$

where s_0 is the initial state, and $a_i \in \alpha(s_i)$ and $s_{i+1} = \delta(s_i, a_{i+1})$ for each i .

Goals, planning, and plans

Goal = propositional formula G on fluents

Planning = find a trace $s_0, a_1, s_1, \dots, a_n, s_n$ such that $s_n \models G$. (PSPACE-complete)

Plan = project traces on actions, i.e., return $a_1, \dots, a_n$.

Planning in Deterministic Domains

Winning states for deterministic domains

Let denote the **set of goal states** as:

$$\llbracket G \rrbracket = \{s \in \mathcal{S} \mid s \models G\}$$

and let's define the **(existential) preimage of a set $\mathcal{E}$** the following function:

$$PreE(\mathcal{E}) = \{s \in \mathcal{S} \mid \exists a \in \alpha(s). \delta(s, a) \in \mathcal{E}\}$$

Compute the set Win of winning states of the domain for goal G , i.e., states from which the agent can reach the goal G , by **least-fixpoint**:

- $Win_0 = \llbracket G \rrbracket$ (the goal states)
- $Win_{i+1} = Win_i \cup PreE(Win_i)$
- $Win = \bigcup_i Win_i$

```

Wold := ∅
W := ⌊G⌋
while (W ≠ Wold) {
    Wold := W
    W := W ∪ PreE(W)
}
return W
  
```

(Computing Win is linear in the number of states in $\mathcal{G}$)

Computing the winning strategy

Let's define $\omega : \mathcal{S} \rightarrow 2^{\mathcal{A}}$ as:

$$\omega(s) = \{a \in \alpha(s) \mid \text{if } s \in Win_{i+1} - Win_i \text{ then } \delta(s, a) \in Win_i\}$$

Every way of restricting $\omega(s)$ to return only one action (chosen arbitrarily) gives a **winning strategy** to reach G , i.e., a **plan**.

Planning in Deterministic Domains

- The kind of plan just computed is called “**Universal Plan**”
 - It is **able to say what to do to win in every single state!**
 - **If** for any reason during the execution of the plan the agent find itself in an **unexpected state**
 - i.e.: the agent expected an effect but s/he got another one

Then the **universal plan knows what to do** to win anyway
- Typically in planning it is sufficient to have a **classical plan** (not a universal one), i.e., a **sequence of actions that from the initial state leads the agent to a state satisfying the goal.**
- To compute **classical plans** there are **better algorithms** (in fact exceptionally fast) - see later in the course.

Planning in AI

STRIPS

STRIPS (Stanford Research Institute Problem Solver)

- **STRIPS** [Fikes and Nilsson (1971)] was originally developed for the pioneering autonomous robot **Shakey**
- STRIPS specifies a **transition system** through a simple use of **logic**
 - **Fluents**
 - **propositional atomic formulas**
 - **Actions** specification:
 - **Preconditions**: just a conjunction of (propositional) fluents
 - **Effects**: a conjunction of positive and negative fluents
 - **Frame**: every fluent not mentioned in the effects remains unchanged
 - **Initial situation** specification
 - A conjunction of fluents (all other fluents are false)
 - **Goal** specification
 - A conjunction of fluents (all other fluents can assume any value)
- STRIPS is **simple** but very **much used** in planning

STRIPS

- A problem in STRIPS is a tuple $P = \langle F, A, I, G \rangle$:
 - F stands for set of all fluents (Boolean vars)
 - A stands for set of all actions (also called operators)
 - $I \subseteq F$ stands for initial situation
 - $G \subseteq F$ stands for goal condition
- Actions $a \in A$ represented by
 - the **Precondition list** $\text{Pre}(a) \subseteq F$ (the fluents that need to be true to execute the action)
 - the **Add list** $\text{Add}(a) \subseteq F$ (the fluents that will become positive)
 - the **Delete list** $\text{Del}(a) \subseteq F$ (the fluents that will become negative)
(the fluents not in $\text{Add}(a)$ and $\text{Del}(a)$ remain unchanged, solving the frame problem)

STRIPS

A STRIPS problem $P = \langle F, A, I, G \rangle$ determines transition system $D = (S, A, \text{delta}, \text{alpha})$ and a goal states $[[G]]$ where:

- The **states** $s \in S$ are collections of fluents from F (valuations over F)
- The **initial state** s_0 is I (initially a fluent in F is true iff it's in I)
- The **possible actions** $\text{alpha}(s)$ are the actions $a \in A$ such that $\text{Pre}(a) \subseteq s$

$$\text{alpha}(s) = \{a \in A \mid \text{Pre}(a) \subseteq s\}$$

- The **next state** is

$$s' = s \setminus \text{Del}(a) \cup \text{Add}(a)$$

- The **goal states** are such that

$$[[G]] = \{s \in S \mid G \subseteq s\}$$

Planning in AI

ADL

ADL (Action Description Language)

- **ADL** [Pednault (1986)] was originally developed as an improved version of STRIPS, also influenced by Situation Calculus, contains important ideas incorporated in Reiter's Situation Calculus Preconditions Axioms and Successor State Axioms
- ADL specifies a **transition system** through a more sophisticated use of **logic**
 - **Fluents**
 - **propositional atomic formulas, or FOL atomic formulas to be instantiated on a finite domain!**
 - **Actions** specification:
 - **Preconditions:** **arbitrary propositional/FOL formula on fluents**
 - **Effects:** a conjunction of positive and negative fluents, but can have **arbitrary propositional/FOL conditions;** effects are **conditional:** when performing action a
 - if **condition** $\text{Cond}_i(a)$ holds now
 - Then **effect** $\text{Add}_i(a), \text{Del}_i(a)$ hold next
$$\text{Cond}_i(a) \sim > \text{Add}_i(a), \text{Del}_i(a) \quad (i=1, \dots, n)$$
 - **Frame:** every fluent not mentioned in the effects remains unchanged
 - **Initial situation** specification
 - A conjunction of fluents (all other fluents are false)
 - **Goal** specification
 - An **arbitrary propositional/FOL formula on fluents**

ADL

An **ADL** problem $P = \langle F, A, I, G \rangle$ determines transition system $D = (S, A, \delta, \alpha)$ and a goal states $[[G]]$, *after having instantiated all FOL formulas (grounding) and obtained a corresponding propositional specification of P* , where :

- The **states** $s \in S$ are collections of fluents from F (valuations over F)
- The **initial state** s_0 is I (initially a fluent in F is true iff it's in I)
- The **possible actions** $\alpha(s)$ are the actions $a \in A$ such that $\text{Pre}(a) \subseteq s$

$$\alpha(s) = \{a \in A \mid s \models \text{Pre}(a)\}$$

- The **next state** is

$$s' = s \setminus (\bigcup_i \text{Del}_i(a) \text{ where } s \models \text{Cond}_i(a)) \cup \bigcup_i \text{Add}_i(a) \text{ where } s \models \text{Cond}_i(a)$$

- The **goal states** are such that

$$[[G]] = \{s \in S \mid s \models G\}$$

ADL

Theorem: Any ADL (and hence STRIPS) domain specification can be (polynomially) expressed into a Reiter's Situation Calculus Basic Action Theory.

Theorem: Any Reiter's Situation Calculus Basic Action Theory, (1) on finite object domain (2) with initial situation description being a DB, can be (polynomially) expressed into ADL (but not in STRIPS).

Theorem: ADL Planning tools can be used for solving planning in Reiter's Situation Calculus Basic Action Theory on finite domain with initial situation description being a DB.